

OTR Spare Parts Calculator - Technical Specification

Document Information

- **Project Name:** OTR Spare Parts Calculator Web Application
 - **Client:** Eagle Manufacturing
 - **Version:** 1.0
 - **Date:** January 14, 2026
 - **Status:** Draft for Development
-

1. System Overview

1.1 Purpose

Transform the existing Excel-based spare parts calculator into a multi-user web application with authentication, scenario management, historical tracking, and PDF export capabilities.

1.2 Key Features

- ☒ Multi-user authentication system
- ☒ Personal calculation scenarios (save/load/update)
- ☒ Historical calculation tracking
- ☒ Real-time calculation with formulas
- ☒ Interactive pie chart visualizations
- ☒ Professional PDF export
- ☒ Responsive dashboard layout
- ☒ English language interface

1.3 Technology Stack

Backend

- **Language:** Python 3.11+
- **Framework:** FastAPI 0.104+
- **Database:** PostgreSQL 15+
- **ORM:** SQLAlchemy 2.0+
- **Authentication:** JWT (PyJWT)
- **PDF Generation:** ReportLab or WeasyPrint
- **Validation:** Pydantic V2

Frontend

- **Framework:** React 18+ with TypeScript
- **UI Library:** Material-UI (MUI) v5 or Tailwind CSS
- **Charts:** Chart.js 4+ or Recharts
- **State Management:** Redux Toolkit or Zustand

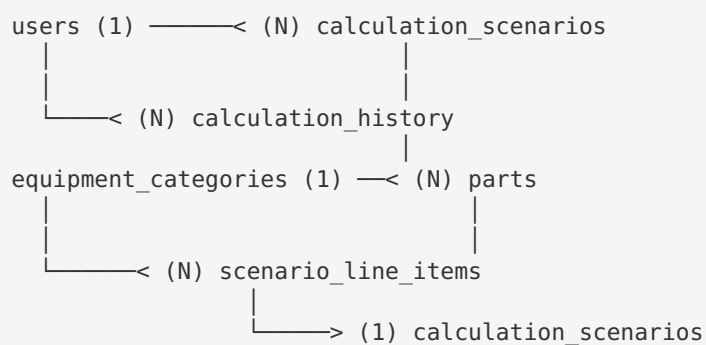
- **HTTP Client:** Axios
- **Build Tool:** Vite

DevOps

- **Containerization:** Docker + Docker Compose
- **CI/CD:** GitHub Actions or GitLab CI
- **Hosting:** AWS/Azure/Heroku
- **Database:** AWS RDS or Azure Database for PostgreSQL

2. Database Schema

2.1 Entity Relationship Diagram



2.2 Table Definitions

Table: users

Primary user authentication and profile information.

```

CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  first_name VARCHAR(100),
  last_name VARCHAR(100),
  company VARCHAR(100),
  is_active BOOLEAN DEFAULT TRUE,
  is_admin BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  last_login TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_username ON users(username);

```

Table: equipment_categories

Master list of equipment types (OTR Debeader, Punch Cutter II, etc.).

```
CREATE TABLE equipment_categories (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) UNIQUE NOT NULL,
  description TEXT,
  display_order INTEGER NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_equipment_display_order ON equipment_categories(display_order);
```

Initial Data:

```
INSERT INTO equipment_categories (name, description, display_order) VALUES
('OTR Debeader', 'Tire debearing equipment spare parts', 1),
('Punch Cutter II', 'Tire cutting and punching equipment parts', 2),
('Titan II', 'Heavy-duty tire processing equipment parts', 3),
('Interchangeable Parts', 'Universal components compatible across equipment', 4);
```

Table: parts

Master catalog of all spare parts.

```
CREATE TABLE parts (
  id SERIAL PRIMARY KEY,
  equipment_category_id INTEGER REFERENCES equipment_categories(id) ON DELETE CASCADE,
  part_name VARCHAR(200) NOT NULL,
  part_code VARCHAR(50) UNIQUE,
  description TEXT,
  part_type VARCHAR(50), -- hydraulic, blade, electronic, mechanical
  default_quantity INTEGER DEFAULT 1,
  default_price DECIMAL(10, 2) NOT NULL,
  unit_of_measure VARCHAR(20) DEFAULT 'unit',
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT positive_quantity CHECK (default_quantity >= 0),
  CONSTRAINT positive_price CHECK (default_price >= 0)
);

CREATE INDEX idx_parts_category ON parts(equipment_category_id);
CREATE INDEX idx_parts_type ON parts(part_type);
CREATE INDEX idx_parts_active ON parts(is_active);
```

Initial Data: (See spare_parts_data_structure.json for complete list)

Table: calculation_scenarios

User-saved calculation scenarios.

```

CREATE TABLE calculation_scenarios (
  id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
  scenario_name VARCHAR(100) NOT NULL,
  description TEXT,
  notes TEXT,
  grand_total DECIMAL(12, 2),
  is_template BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT user_scenario_unique UNIQUE (user_id, scenario_name)
);

CREATE INDEX idx_scenarios_user ON calculation_scenarios(user_id);
CREATE INDEX idx_scenarios_created ON calculation_scenarios(created_at DESC);

```

Table: scenario_line_items

Individual parts in each calculation scenario.

```

CREATE TABLE scenario_line_items (
  id SERIAL PRIMARY KEY,
  scenario_id INTEGER REFERENCES calculation_scenarios(id) ON DELETE CASCADE,
  part_id INTEGER REFERENCES parts(id) ON DELETE CASCADE,
  equipment_category_id INTEGER REFERENCES equipment_categories(id),
  quantity INTEGER NOT NULL,
  unit_price DECIMAL(10, 2) NOT NULL,
  line_total DECIMAL(12, 2) GENERATED ALWAYS AS (quantity * unit_price) STORED,
  notes TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT positive_quantity CHECK (quantity >= 0),
  CONSTRAINT positive_price CHECK (unit_price >= 0)
);

CREATE INDEX idx_line_items_scenario ON scenario_line_items(scenario_id);
CREATE INDEX idx_line_items_part ON scenario_line_items(part_id);
CREATE INDEX idx_line_items_category ON scenario_line_items(equipment_category_id);

```

Table: calculation_history

Historical log of all calculations performed.

```

CREATE TABLE calculation_history (
  id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users(id) ON DELETE SET NULL,
  scenario_id INTEGER REFERENCES calculation_scenarios(id) ON DELETE SET NULL,
  scenario_name VARCHAR(100),
  grand_total DECIMAL(12, 2),
  calculation_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  snapshot_json JSONB, -- Full calculation snapshot
  pdf_url VARCHAR(500), -- Path to generated PDF
  ip_address INET,
  user_agent TEXT
);

CREATE INDEX idx_history_user ON calculation_history(user_id);
CREATE INDEX idx_history_date ON calculation_history(calculation_date DESC);
CREATE INDEX idx_history_scenario ON calculation_history(scenario_id);

```

3. Backend API Specification

3.1 Base URL

```
Development: http://localhost:8000/api/v1
Production: https://api.eagle-spareparts.com/api/v1
```

3.2 Authentication

All authenticated endpoints require JWT token in header:

```
Authorization: Bearer <token>
```

POST /auth/register

Register a new user account.

Request Body:

```
{
  "username": "john.doe",
  "email": "john.doe@eagle.com",
  "password": "SecurePassword123!",
  "first_name": "John",
  "last_name": "Doe",
  "company": "Eagle Manufacturing"
}
```

Response (201):

```
{
  "id": 1,
  "username": "john.doe",
  "email": "john.doe@eagle.com",
  "first_name": "John",
  "last_name": "Doe",
  "created_at": "2026-01-14T10:30:00Z"
}
```

Validation Rules:

- Username: 3-50 characters, alphanumeric + underscore
- Email: Valid email format
- Password: Min 8 characters, 1 uppercase, 1 number, 1 special char

POST /auth/login

Authenticate user and receive JWT token.

Request Body:

```
{
  "username": "john.doe",
  "password": "SecurePassword123!"
}
```

Response (200):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "expires_in": 3600,
  "user": {
    "id": 1,
    "username": "john.doe",
    "email": "john.doe@eagle.com",
    "first_name": "John",
    "last_name": "Doe"
  }
}
```

POST /auth/refresh

Refresh expired JWT token.

Request Body:

```
{
  "refresh_token": "..."
}
```

Response (200):

```
{
  "access_token": "new_token...",
  "expires_in": 3600
}
```

3.3 Equipment & Parts APIs

GET /equipment-categories

Retrieve all equipment categories.

Response (200):

```
{
  "data": [
    {
      "id": 1,
      "name": "OTR Debeader",
      "description": "Tire debanding equipment spare parts",
      "display_order": 1,
      "is_active": true
    },
    ...
  ]
}
```

GET /equipment-categories/{id}/parts

Retrieve all parts for a specific equipment category.

Response (200):

```
{
  "data": [
    {
      "id": 1,
      "part_name": "Hydraulic Filter",
      "part_code": "HF-001",
      "part_type": "hydraulic",
      "default_quantity": 2,
      "default_price": 181.40,
      "unit_of_measure": "unit"
    },
    ...
  ]
}
```

GET /parts

Retrieve all parts with optional filtering.

Query Parameters:

- `equipment_category_id` (optional): Filter by category
- `part_type` (optional): Filter by type (hydraulic, blade, etc.)
- `search` (optional): Search in part name
- `is_active` (optional): Filter active/inactive parts

Response (200):

```
{
  "data": [
    {
      "id": 1,
      "equipment_category_id": 1,
      "equipment_category_name": "OTR Debeader",
      "part_name": "Hydraulic Filter",
      "part_code": "HF-001",
      "part_type": "hydraulic",
      "default_quantity": 2,
      "default_price": 181.40,
      "description": "High-pressure hydraulic filter",
      "is_active": true
    },
    ...
  ],
  "total": 14,
  "page": 1,
  "page_size": 50
}
```

GET /parts/{id}

Retrieve a single part by ID.

Response (200):

```
{
  "id": 1,
  "equipment_category_id": 1,
  "equipment_category_name": "OTR Debeader",
  "part_name": "Hydraulic Filter",
  "part_code": "HF-001",
  "part_type": "hydraulic",
  "default_quantity": 2,
  "default_price": 181.40,
  "unit_of_measure": "unit",
  "description": "High-pressure hydraulic filter",
  "is_active": true,
  "created_at": "2026-01-01T00:00:00Z",
  "updated_at": "2026-01-01T00:00:00Z"
}
```

3.4 Calculation Scenarios APIs

POST /scenarios

Create a new calculation scenario.

Request Body:

```
{
  "scenario_name": "Q1 2026 Maintenance Order",
  "description": "Quarterly spare parts order",
  "notes": "Rush order for OTR Debeader",
  "line_items": [
    {
      "part_id": 1,
      "quantity": 2,
      "unit_price": 181.40
    },
    {
      "part_id": 3,
      "quantity": 2,
      "unit_price": 607.23
    }
  ]
}
```

Response (201):

```
{
  "id": 123,
  "scenario_name": "Q1 2026 Maintenance Order",
  "description": "Quarterly spare parts order",
  "notes": "Rush order for OTR Debeader",
  "grand_total": 1577.26,
  "line_items": [
    {
      "id": 1001,
      "part_id": 1,
      "part_name": "Hydraulic Filter",
      "equipment_category_name": "OTR Debeader",
      "quantity": 2,
      "unit_price": 181.40,
      "line_total": 362.80
    },
    {
      "id": 1002,
      "part_id": 3,
      "part_name": "Cutter Blade",
      "equipment_category_name": "OTR Debeader",
      "quantity": 2,
      "unit_price": 607.23,
      "line_total": 1214.46
    }
  ],
  "section_totals": [
    {
      "equipment_category_id": 1,
      "equipment_category_name": "OTR Debeader",
      "subtotal": 1577.26,
      "percentage": 100.0
    }
  ],
  "created_at": "2026-01-14T10:45:00Z",
  "updated_at": "2026-01-14T10:45:00Z"
}
```

GET /scenarios

Retrieve all scenarios for the authenticated user.

Query Parameters:

- `page` (default: 1)
- `page_size` (default: 20)
- `sort_by` (default: "created_at")
- `sort_order` (default: "desc")
- `search` (optional): Search in scenario name

Response (200):

```
{
  "data": [
    {
      "id": 123,
      "scenario_name": "Q1 2026 Maintenance Order",
      "description": "Quarterly spare parts order",
      "grand_total": 1577.26,
      "item_count": 2,
      "created_at": "2026-01-14T10:45:00Z",
      "updated_at": "2026-01-14T10:45:00Z"
    },
    ...
  ],
  "total": 45,
  "page": 1,
  "page_size": 20,
  "total_pages": 3
}
```

GET /scenarios/{id}

Retrieve a specific scenario with full details.

Response (200): (Same structure as POST response)

PUT /scenarios/{id}

Update an existing scenario.

Request Body: (Same as POST)

Response (200): (Same as POST response)

DELETE /scenarios/{id}

Delete a scenario.

Response (204): No content

3.5 Calculation APIs**POST /calculate**

Perform a calculation without saving (for preview).

Request Body:

```
{
  "line_items": [
    {
      "part_id": 1,
      "quantity": 2,
      "unit_price": 181.40
    },
    ...
  ]
}
```

Response (200):

```
{
  "grand_total": 13096.22,
  "line_items": [
    {
      "part_id": 1,
      "part_name": "Hydraulic Filter",
      "equipment_category_id": 1,
      "equipment_category_name": "OTR Debeader",
      "quantity": 2,
      "unit_price": 181.40,
      "line_total": 362.80
    },
    ...
  ],
  "section_totals": [
    {
      "equipment_category_id": 1,
      "equipment_category_name": "OTR Debeader",
      "subtotal": 2841.79,
      "percentage": 21.70,
      "item_count": 5
    },
    {
      "equipment_category_id": 2,
      "equipment_category_name": "Punch Cutter II",
      "subtotal": 3301.97,
      "percentage": 25.21,
      "item_count": 4
    },
    {
      "equipment_category_id": 3,
      "equipment_category_name": "Titan II",
      "subtotal": 6296.56,
      "percentage": 48.07,
      "item_count": 4
    },
    {
      "equipment_category_id": 4,
      "equipment_category_name": "Interchangeable Parts",
      "subtotal": 655.90,
      "percentage": 5.01,
      "item_count": 1
    }
  ],
  "calculated_at": "2026-01-14T10:50:00Z"
}
```

3.6 History APIs

GET /history

Retrieve calculation history for the authenticated user.

Query Parameters:

- `page` (default: 1)
- `page_size` (default: 20)
- `date_from` (optional): ISO 8601 date
- `date_to` (optional): ISO 8601 date
- `scenario_id` (optional): Filter by scenario

Response (200):

```
{
  "data": [
    {
      "id": 5001,
      "scenario_id": 123,
      "scenario_name": "Q1 2026 Maintenance Order",
      "grand_total": 1577.26,
      "calculation_date": "2026-01-14T10:45:00Z",
      "pdf_url": "/exports/calc_5001.pdf"
    },
    ...
  ],
  "total": 150,
  "page": 1,
  "page_size": 20
}
```

GET /history/{id}

Retrieve a specific calculation from history.

Response (200):

```
{
  "id": 5001,
  "user_id": 1,
  "scenario_id": 123,
  "scenario_name": "Q1 2026 Maintenance Order",
  "grand_total": 1577.26,
  "calculation_date": "2026-01-14T10:45:00Z",
  "snapshot": {
    "line_items": [...],
    "section_totals": [...],
    "grand_total": 1577.26
  },
  "pdf_url": "/exports/calc_5001.pdf"
}
```

3.7 PDF Export API

POST /export/pdf

Generate and download a PDF report.

Request Body:

```
{
  "scenario_id": 123,
  "include_chart": true,
  "include_notes": true,
  "company_logo": true
}
```

Response (200):

- Content-Type: application/pdf
- Content-Disposition: attachment; filename="OTR_Spare_Parts_Scenario_123.pdf"
- Binary PDF data

Or for async generation:**Response (202):**

```
{
  "job_id": "pdf-gen-xyz123",
  "status": "processing",
  "estimated_completion": "2026-01-14T10:55:00Z"
}
```

GET /export/pdf/status/{job_id}

Check PDF generation status.

Response (200):

```
{
  "job_id": "pdf-gen-xyz123",
  "status": "completed",
  "pdf_url": "/exports/calc_5001.pdf",
  "created_at": "2026-01-14T10:54:30Z"
}
```

4. Frontend Architecture

4.1 Component Structure

```

src/
├── components/
│   ├── auth/
│   │   ├── LoginForm.tsx
│   │   ├── RegisterForm.tsx
│   │   └── ProtectedRoute.tsx
│   ├── calculator/
│   │   ├── CalculatorDashboard.tsx
│   │   ├── InputPanel.tsx
│   │   ├── ResultsPanel.tsx
│   │   ├── EquipmentSection.tsx
│   │   ├── PartLineItem.tsx
│   │   └── GrandTotalDisplay.tsx
│   ├── charts/
│   │   ├── CostDistributionPie.tsx
│   │   └── ChartExport.tsx
│   ├── scenarios/
│   │   ├── ScenarioList.tsx
│   │   ├── ScenarioCard.tsx
│   │   ├── SaveScenarioDialog.tsx
│   │   └── LoadScenarioDialog.tsx
│   ├── history/
│   │   ├── HistoryTable.tsx
│   │   └── HistoryDetail.tsx
│   ├── pdf/
│   │   └── PDFExportButton.tsx
│   └── common/
│       ├── Header.tsx
│       ├── Sidebar.tsx
│       ├── Footer.tsx
│       └── LoadingSpinner.tsx
├── pages/
│   ├── LoginPage.tsx
│   ├── RegisterPage.tsx
│   ├── DashboardPage.tsx
│   ├── CalculatorPage.tsx
│   ├── ScenariosPage.tsx
│   └── HistoryPage.tsx
├── services/
│   ├── api.ts
│   ├── auth.service.ts
│   ├── calculator.service.ts
│   ├── scenarios.service.ts
│   └── history.service.ts
├── store/
│   ├── authSlice.ts
│   ├── calculatorSlice.ts
│   ├── partsSlice.ts
│   └── store.ts
├── utils/
│   ├── calculations.ts
│   ├── formatters.ts
│   └── validators.ts
├── types/
│   ├── auth.types.ts
│   ├── calculator.types.ts
│   └── api.types.ts
├── hooks/
│   ├── useAuth.ts
│   ├── useCalculator.ts
│   └── useScenarios.ts
└── App.tsx

```

4.2 Key Components

CalculatorDashboard.tsx

Main calculator interface with split-panel layout.

```
interface CalculatorDashboardProps {}

const CalculatorDashboard: React.FC = () => {
  return (
    <Box sx={{ display: 'flex', height: '100vh' }}>
      <Box sx={{ flex: '0 0 50%', overflow: 'auto', p: 3 }}>
        <InputPanel />
      </Box>
      <Box sx={{ flex: '0 0 50%', overflow: 'auto', p: 3, bgcolor: '#f5f5f5' }}>
        <ResultsPanel />
      </Box>
    </Box>
  );
};
```

InputPanel.tsx

Left panel with all input controls.

```
interface InputPanelProps {}

const InputPanel: React.FC = () => {
  const { equipmentCategories, parts, updateQuantity, updatePrice } = useCalculator();

  return (
    <Box>
      <Typography variant="h4">New Calculation</Typography>

      <Box sx={{ my: 2 }}>
        <TextField label="Scenario Name" fullWidth />
        <ButtonGroup>
          <Button variant="contained">Save Scenario</Button>
          <Button variant="outlined">Load Scenario</Button>
        </ButtonGroup>
      </Box>

      {equipmentCategories.map(category => (
        <EquipmentSection key={category.id} category={category} />
      ))}

      <Box sx={{ mt: 3 }}>
        <Button variant="contained" color="primary" size="large">
          Calculate
        </Button>
        <Button variant="outlined" size="large">
          Reset
        </Button>
      </Box>
    </Box>
  );
};
```

ResultsPanel.tsx

Right panel displaying calculation results.

```

interface ResultsPanelProps {}

const ResultsPanel: React.FC = () => {
  const { sectionTotals, grandTotal } = useCalculator();

  return (
    <Box>
      <Typography variant="h4">Calculation Results</Typography>

      <Box sx={{ my: 3 }}>
        <Typography variant="h6">Equipment Breakdown:</Typography>
        {sectionTotals.map(section => (
          <Box key={section.id} sx={{ display: 'flex', justifyContent: 'space-
between', my: 1 }}>
            <Typography>{section.name}</Typography>
            <Typography fontWeight="bold">${section.subtotal.toFixed(2)}</Typography>
          </Box>
        ))}
      </Box>

      <Divider sx={{ my: 3 }} />

      <GrandTotalDisplay total={grandTotal} />

      <Box sx={{ my: 3 }}>
        <Typography variant="h6">Cost Distribution</Typography>
        <CostDistributionPie data={sectionTotals} />
      </Box>

      <Box sx={{ mt: 3 }}>
        <Button variant="contained" color="secondary" startIcon={<PictureAsPdfIcon />}
>
          Export PDF
        </Button>
        <Button variant="outlined" startIcon={<SaveIcon />}>
          Save Calculation
        </Button>
      </Box>
    </Box>
  );
};

```

4.3 State Management (Redux)

`calculatorSlice.ts`

```

interface CalculatorState {
  equipmentCategories: EquipmentCategory[];
  parts: Part[];
  lineItems: LineItem[];
  sectionTotals: SectionTotal[];
  grandTotal: number;
  loading: boolean;
  error: string | null;
}

const initialState: CalculatorState = {
  equipmentCategories: [],
  parts: [],
  lineItems: [],
  sectionTotals: [],
  grandTotal: 0,
  loading: false,
  error: null,
};

const calculatorSlice = createSlice({
  name: 'calculator',
  initialState,
  reducers: {
    setEquipmentCategories: (state, action: PayloadAction<EquipmentCategory[]>) => {
      state.equipmentCategories = action.payload;
    },
    setParts: (state, action: PayloadAction<Part[]>) => {
      state.parts = action.payload;
    },
    updateLineItem: (state, action: PayloadAction<{ partId: number; quantity: number;
unitPrice: number }>) => {
      const existingIndex = state.lineItems.findIndex(item => item.partId === ac-
tion.payload.partId);
      if (existingIndex >= 0) {
        state.lineItems[existingIndex] = {
          ...state.lineItems[existingIndex],
          ...action.payload,
          lineTotal: action.payload.quantity * action.payload.unitPrice,
        };
      } else {
        state.lineItems.push({
          partId: action.payload.partId,
          quantity: action.payload.quantity,
          unitPrice: action.payload.unitPrice,
          lineTotal: action.payload.quantity * action.payload.unitPrice,
        });
      }
    },
    calculateTotals: (state) => {
      // Calculate section totals
      const sectionTotalsMap = new Map<number, number>();
      state.lineItems.forEach(item => {
        const part = state.parts.find(p => p.id === item.partId);
        if (part) {
          const currentTotal = sectionTotalsMap.get(part.equipmentCategoryId) || 0;
          sectionTotalsMap.set(part.equipmentCategoryId, currentTotal + item.lineTotal
);
        }
      });

      // Convert to array

```

```

    state.sectionTotals = Array.from(sectionTotalsMap.entries()).map(([categoryId, subtotal]) => {
      const category = state.equipmentCategories.find(c => c.id === categoryId);
      return {
        equipmentCategoryId: categoryId,
        equipmentCategoryName: category?.name || '',
        subtotal,
        percentage: 0, // Will be calculated after grand total
      };
    });

    // Calculate grand total
    state.grandTotal = state.sectionTotals.reduce((sum, section) => sum + section.subtotal, 0);

    // Calculate percentages
    state.sectionTotals = state.sectionTotals.map(section => ({
      ...section,
      percentage: state.grandTotal > 0 ? (section.subtotal / state.grandTotal) *
100 : 0,
    }));
  },
  resetCalculator: (state) => {
    state.lineItems = [];
    state.sectionTotals = [];
    state.grandTotal = 0;
  },
});

export const { setEquipmentCategories, setParts, updateLineItem, calculateTotals, resetCalculator } = calculatorSlice.actions;
export default calculatorSlice.reducer;

```

4.4 Calculation Logic (utils/calculations.ts)

```

export interface LineItem {
  partId: number;
  partName: string;
  equipmentCategoryId: number;
  equipmentCategoryName: string;
  quantity: number;
  unitPrice: number;
  lineTotal: number;
}

export interface SectionTotal {
  equipmentCategoryId: number;
  equipmentCategoryName: string;
  subtotal: number;
  percentage: number;
  itemCount: number;
}

export const calculateLineTotal = (quantity: number, unitPrice: number): number => {
  return parseFloat((quantity * unitPrice).toFixed(2));
};

export const calculateSectionTotals = (lineItems: LineItem[]): SectionTotal[] => {
  const sectionsMap = new Map<number, { name: string; total: number; count: number }>();

  lineItems.forEach(item => {
    const existing = sectionsMap.get(item.equipmentCategoryId);
    if (existing) {
      existing.total += item.lineTotal;
      existing.count += 1;
    } else {
      sectionsMap.set(item.equipmentCategoryId, {
        name: item.equipmentCategoryName,
        total: item.lineTotal,
        count: 1,
      });
    }
  });

  const grandTotal = Array.from(sectionsMap.values()).reduce((sum, section) => sum + section.total, 0);

  return Array.from(sectionsMap.entries()).map(([id, data]) => ({
    equipmentCategoryId: id,
    equipmentCategoryName: data.name,
    subtotal: parseFloat(data.total.toFixed(2)),
    percentage: grandTotal > 0 ? parseFloat(((data.total / grandTotal) * 100).toFixed(2)) : 0,
    itemCount: data.count,
  }));
};

export const calculateGrandTotal = (sectionTotals: SectionTotal[]): number => {
  return parseFloat(sectionTotals.reduce((sum, section) => sum + section.subtotal, 0).toFixed(2));
};

```

5. PDF Export Specification

5.1 PDF Layout

[LOGO]	OTR SPARE PARTS CALCULATION		
Scenario: Q1 2026 Maintenance Order			
Date: January 14, 2026			
Prepared by: John Doe (john.doe@eagle.com)			
Company: Eagle Manufacturing			
OTR DEBEADER			
Part	Qty	Price/Unit	Total
Hydraulic Filter	2	\$181.40	\$362.80
Limit Switch	2	\$505.23	\$1,010.46
...			
Subtotal:			\$2,841.79
PUNCH CUTTER II			
...			
[Similar sections for Titan II and Interchangeable Parts]			
COST DISTRIBUTION			
[Pie Chart Image]			
GRAND TOTAL : \$13,096.22			
Notes: Rush order for OTR Debeader			
Generated by Eagle OTR Spare Parts Calculator			
Page 1 of 1			

5.2 PDF Generation (Python/ReportLab)

```

from reportlab.lib.pagesizes import letter, A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, Paragraph,
Spacer, Image, PageBreak
from reportlab.lib import colors
from datetime import datetime

def generate_calculation_pdf(scenario_data, user_data, output_path):
    """
    Generate a PDF report for a calculation scenario.
    """
    doc = SimpleDocTemplate(output_path, pagesize=letter,
                           rightMargin=72, leftMargin=72,
                           topMargin=72, bottomMargin=18)

    # Container for elements
    elements = []

    # Styles
    styles = getSampleStyleSheet()
    title_style = ParagraphStyle(
        'CustomTitle',
        parent=styles['Heading1'],
        fontSize=24,
        textColor=colors.HexColor('#1976d2'),
        spaceAfter=30,
        alignment=1 # Center
    )

    # Title
    title = Paragraph("OTR SPARE PARTS CALCULATION", title_style)
    elements.append(title)
    elements.append(Spacer(1, 12))

    # Metadata
    meta_data = [
        ["Scenario:", scenario_data['scenario_name']],
        ["Date:", datetime.now().strftime("%B %d, %Y")],
        ["Prepared by:", f"{user_data['first_name']} {user_data['last_name']} ({user_data['email']}")],
        ["Company:", user_data.get('company', 'Eagle Manufacturing')]
    ]

    meta_table = Table(meta_data, colWidths=[1.5*inch, 4*inch])
    meta_table.setStyle(TableStyle([
        ('FONTNAME', (0, 0), (0, -1), 'Helvetica-Bold'),
        ('FONTSIZE', (0, 0), (-1, -1), 10),
        ('BOTTOMPADDING', (0, 0), (-1, -1), 6),
    ]))

    elements.append(meta_table)
    elements.append(Spacer(1, 24))

    # Equipment sections
    for section in scenario_data['section_totals']:
        # Section header
        section_header = Paragraph(section['equipment_category_name'], styles['Heading2'])
        elements.append(section_header)
        elements.append(Spacer(1, 12))

```

```

# Get line items for this section
section_items = [item for item in scenario_data['line_items']
                  if item['equipment_category_id'] == section['equipment_category_id']]

# Create table data
table_data = [['Part', 'Quantity', 'Price/Unit', 'Total']]
for item in section_items:
    table_data.append([
        item['part_name'],
        str(item['quantity']),
        f"${item['unit_price']:.2f}",
        f"${item['line_total']:.2f}"
    ])

# Subtotal row
table_data.append(['', '', 'Subtotal:', f"${section['subtotal']:.2f}"])

# Create table
item_table = Table(table_data, colWidths=[3*inch, 0.8*inch, 1*inch, 1*inch])
item_table.setStyle(TableStyle([
    ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
    ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
    ('ALIGN', (1, 0), (-1, -1), 'RIGHT'),
    ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
    ('FONTSIZE', (0, 0), (-1, 0), 12),
    ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
    ('BACKGROUND', (0, 1), (-1, -2), colors.beige),
    ('GRID', (0, 0), (-1, -2), 1, colors.black),
    ('FONTNAME', (2, -1), (-1, -1), 'Helvetica-Bold'),
    ('BACKGROUND', (2, -1), (-1, -1), colors.lightgrey),
]))

elements.append(item_table)
elements.append(Spacer(1, 24))

# Grand Total
grand_total_para = Paragraph(
    f"<para align=center fontSize=18><b>GRAND TOTAL: $
{scenario_data['grand_total']:.2f}</b></para>",
    styles['Normal'])
)
elements.append(Spacer(1, 12))
elements.append(grand_total_para)
elements.append(Spacer(1, 24))

# Notes
if scenario_data.get('notes'):
    notes_para = Paragraph(f"<b>Notes:</b> {scenario_data['notes']}", styles['Normal'])
    elements.append(notes_para)

# Build PDF
doc.build(elements)

return output_path

```

6. Security Considerations

6.1 Authentication & Authorization

- JWT tokens with expiration (1 hour access, 7 days refresh)
- Password hashing: bcrypt with cost factor 12
- Rate limiting: 5 failed login attempts → 15-minute logout
- Session management: Store refresh tokens securely
- CORS configuration: Whitelist allowed origins

6.2 Input Validation

- Sanitize all user inputs (scenario names, notes)
- Validate numeric inputs (quantity, price) with range checks
- SQL injection prevention: Use parameterized queries (ORM)
- XSS prevention: Escape HTML in user-generated content

6.3 Data Protection

- HTTPS/TLS for all communications
- Database encryption at rest
- Sensitive data encryption (if required by regulations)
- Regular security audits and penetration testing
- Backup strategy: Daily backups with 30-day retention

6.4 API Security

- API key/token authentication for all endpoints
 - Request size limits (prevent DoS)
 - Rate limiting per user/IP
 - Input schema validation (Pydantic)
 - CSRF token for state-changing operations
-

7. Testing Strategy

7.1 Backend Testing

Unit Tests

```
# tests/test_calculations.py
def test_line_total_calculation():
    assert calculate_line_total(2, 181.40) == 362.80
    assert calculate_line_total(2, 607.23) == 1214.46

def test_section_total_calculation():
    line_items = [
        LineItem(part_id=1, quantity=2, unit_price=181.40, category_id=1),
        LineItem(part_id=2, quantity=2, unit_price=505.23, category_id=1),
    ]
    section_totals = calculate_section_totals(line_items)
    assert section_totals[0].subtotal == 1372.86

def test_grand_total_calculation():
    section_totals = [
        SectionTotal(category_id=1, subtotal=2841.79),
        SectionTotal(category_id=2, subtotal=3301.97),
    ]
    assert calculate_grand_total(section_totals) == 6143.76
```

Integration Tests

```
# tests/test_api_scenarios.py
def test_create_scenario(client, auth_token):
    response = client.post(
        '/api/v1/scenarios',
        json={
            'scenario_name': 'Test Scenario',
            'line_items': [...]
        },
        headers={'Authorization': f'Bearer {auth_token}'}
    )
    assert response.status_code == 201
    assert 'grand_total' in response.json()
```

7.2 Frontend Testing

Component Tests (Jest + React Testing Library)

```
// components/__tests__/InputPanel.test.tsx
describe('InputPanel', () => {
  it('should render all equipment sections', () => {
    render(<InputPanel />);
    expect(screen.getByText('OTR Debeader')).toBeInTheDocument();
    expect(screen.getByText('Punch Cutter II')).toBeInTheDocument();
  });

  it('should update quantity on input change', () => {
    render(<InputPanel />);
    const qtyInput = screen.getByLabelText('Quantity');
    fireEvent.change(qtyInput, { target: { value: '5' } });
    expect(qtyInput).toHaveValue(5);
  });
});
```

End-to-End Tests (Playwright/Cypress)

```
// e2e/calculator.spec.ts
test('complete calculation workflow', async ({ page }) => {
  await page.goto('http://localhost:3000');
  await page.fill('[name="username"]', 'testuser');
  await page.fill('[name="password"]', 'TestPass123!');
  await page.click('button[type="submit"]');

  await page.click('text=New Calculation');
  await page.fill('[name="quantity-1"]', '2');
  await page.fill('[name="price-1"]', '181.40');
  await page.click('text=Calculate');

  await expect(page.locator('.grand-total')).toContainText('$362.80');
});
```

8. Deployment Strategy

8.1 Docker Configuration

Dockerfile (Backend)

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dockerfile (Frontend)

```
FROM node:18-alpine AS builder

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

FROM nginx:alpine

COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/nginx.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

docker-compose.yml

```
version: '3.8'

services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: otr_spareparts
      POSTGRES_USER: otr_user
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  backend:
    build: ./backend
    environment:
      DATABASE_URL: postgresql://otr_user:${DB_PASSWORD}@db:5432/otr_spareparts
      JWT_SECRET: ${JWT_SECRET}
    depends_on:
      - db
    ports:
      - "8000:8000"
    volumes:
      - ./backend:/app
      - ./exports:/app/exports

  frontend:
    build: ./frontend
    ports:
      - "80:80"
    depends_on:
      - backend

volumes:
  postgres_data:
```

8.2 CI/CD Pipeline (GitHub Actions)

```
# .github/workflows/deploy.yml
name: Build and Deploy

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run Backend Tests
        run: |
          cd backend
          pip install -r requirements.txt
          pytest
      - name: Run Frontend Tests
        run: |
          cd frontend
          npm ci
          npm test

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Deploy to Production
        run: |
          # Deploy commands here
```

9. Performance Optimization

9.1 Backend Optimizations

- Database connection pooling (SQLAlchemy pool_size=20)
- Query optimization with indexes
- Caching frequently accessed data (Redis)
- Async I/O for PDF generation
- Pagination for large datasets

9.2 Frontend Optimizations

- Code splitting and lazy loading
 - React.memo for expensive components
 - Virtual scrolling for long lists
 - Debouncing for input handlers
 - Service Worker for offline capability
 - CDN for static assets
-

10. Monitoring & Logging

10.1 Logging Strategy

- Application logs: INFO level in production
- Access logs: All API requests
- Error logs: Stack traces with context
- Audit logs: User actions (create/update/delete)

10.2 Monitoring Tools

- **APM:** New Relic or DataDog
 - **Error Tracking:** Sentry
 - **Uptime Monitoring:** Pingdom or UptimeRobot
 - **Database Monitoring:** pgAdmin or CloudWatch
-

11. Future Enhancements

11.1 Phase 2 Features

- Multi-currency support
- Discount/promotion codes
- Bulk import from CSV/Excel
- Email notifications for saved scenarios
- Collaborative scenarios (team sharing)
- Advanced search and filtering
- Mobile app (React Native)

11.2 Phase 3 Features

- Integration with ERP systems
 - Automated purchase order generation
 - Inventory level tracking
 - Predictive maintenance recommendations
 - Multi-language support
 - Advanced analytics dashboard
 - API for third-party integrations
-

12. Project Timeline

Phase 1: Foundation (4 weeks)

- Week 1: Database setup, API skeleton
- Week 2: Authentication system, CRUD endpoints
- Week 3: Frontend setup, basic calculator UI
- Week 4: Integration, testing

Phase 2: Core Features (4 weeks)

- Week 5-6: Scenario management, history
- Week 7: PDF export, charts
- Week 8: Polish, bug fixes, testing

Phase 3: Launch (2 weeks)

- Week 9: User acceptance testing
- Week 10: Deployment, documentation

Total: 10 weeks

Appendix

A. Formula Reference

Line Total = Quantity × Unit Price
Section Subtotal = SUM(Line Totals in Section)
Grand Total = SUM(All Section Subtotals)
Section Percentage = (Section Subtotal / Grand Total) × 100

B. API Error Codes

Code	Message	Description
400	Bad Request	Invalid input data
401	Unauthorized	Missing or invalid token
403	Forbidden	Insufficient permissions
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate resource
422	Unprocessable Entity	Validation error
500	Internal Server Error	Server error

C. Environment Variables

```
# Backend
DATABASE_URL=postgresql://user:pass@localhost:5432/db
JWT_SECRET=your-secret-key
JWT_ALGORITHM=HS256
JWT_EXPIRATION=3600
CORS_ORIGINS=http://localhost:3000
PDF_EXPORT_DIR=/app/exports

# Frontend
REACT_APP_API_URL=http://localhost:8000/api/v1
REACT_APP_ENVIRONMENT=development
```

End of Technical Specification